FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Sagpang, Edriane P.
Section:	AN43
Professor:	Ms. Tan
Date:	

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]
Rule-Based Reasoning Actions: - Turn on the aircon - Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

KR **code:**

"""

TA3 - Implementing Knowledge Representation, Rule-Based Reasoning (RBR), and Case-Based Reasoning (CBR)

Use case: Laptop Troubleshooting Assistant
Both reasoning systems try to answer the same question:

```
"What should we do about this laptop problem?"
"""

import sys

# ============================================================
# PART 1 - KNOWLEDGE REPRESENTATION (KR)
# Task 1A - Knowledge base stored as a Python dictionary
# 6 facts about one object (a laptop), grouped into 3 categories.
# ============================================================
facts = {
    "power": {                  # Category 1: power
        "turns_on": True,
        "charger_connected": False,
        "battery_percent": 12,
    },
    "performance": {            # Category 2: performance
        "cpu_temperature": 92,      # degrees Celsius
        "is_slow": True,
    },
    "status": {                 # Category 3: status / symptoms
        "fan_noise": True,
    },
}


def show_knowledge_base(kb):
    """Print the knowledge base in a readable format."""
    print("Knowledge Base (Facts about the Laptop):")
    for category, items in kb.items():
        print(f"  [{category.upper()}]")
        for name, value in items.items():
            print(f"    - {name}: {value}")
show_knowledge_base(facts)
```

```
edrianesagpang@192 TA4 % /usr/local/bin/python3 /Users/edrianesagpang/Documents/TA4/TA4kr.py
Knowledge Base (Facts about the Laptop):
  [POWER]
    - turns_on: True
    - charger_connected: False
    - battery_percent: 12
  [PERFORMANCE]
    - cpu_temperature: 92
    - is_slow: True
  [STATUS]
    - fan_noise: True
edrianesagpang@192 TA4 %
```

RBR **code:**

```
# ====================================================================
# PART 2 - RULE-BASED REASONING (RBR)
# Task 2A - IF-THEN rules written as Python functions.
# Each rule returns an action if its condition is true, else None.
# ====================================================================
from TA4kr import facts, show_knowledge_base
def rule_low_battery(f):
    # IF battery is below 20% AND charger is not connected THEN plug in charger
    if f["power"]["battery_percent"] < 20 and not f["power"]["charger_connected"]:
        return "Plug in the charger"


def rule_no_power(f):
    # IF laptop does not turn on AND charger is connected THEN check the cable
    if not f["power"]["turns_on"] and f["power"]["charger_connected"]:
        return "Check the power cable and charger port"


def rule_overheating(f):
    # IF CPU temperature is above 85 C THEN clean vents and cooling fan
    if f["performance"]["cpu_temperature"] > 85:
        return "Clean the air vents and cooling fan"


def rule_slow_and_noisy(f):
    # IF laptop is slow AND fan is noisy THEN close heavy apps / check for malware
    if f["performance"]["is_slow"] and f["status"]["fan_noise"]:
        return "Close heavy programs and run a malware scan"


def rule_critical_battery_slow(f):
    # IF laptop is slow AND battery is under 20% THEN enable battery saver
    if f["performance"]["is_slow"] and f["power"]["battery_percent"] < 20:
        return "Turn on battery saver mode"
```

```
# The rule base: a list of all rule functions
rules = [
    rule_low_battery,
    rule_no_power,
    rule_overheating,
    rule_slow_and_noisy,
    rule_critical_battery_slow,
]


# Task 2B - Inference Engine
def inference_engine(kb, rule_list):
    """
    1. Reads the knowledge base
    2. Checks each rule
    3. Returns all actions inferred
    """
    actions = []
    for rule in rule_list:
        result = rule(kb)          # Check the rule against the facts
        if result is not None:     # Rule fired
            actions.append(result)
    return actions


# Task 2C - Display Output
def run_rbr():
    print("=" * 55)
    print("RULE-BASED REASONING (RBR)")
    print("=" * 55)
    show_knowledge_base(facts)
    print()
    actions = inference_engine(facts, rules)
    print("Rule-Based Reasoning Actions:")
    if actions:
        for action in actions:
            print(f"- {action}")
    else:
        print("- No rule matched the given facts")
    print()

if __name__ == "__main__":
```

run_rbr()

```
edrianesagpang@192 TA4 % python3 TA4rbr.py
Knowledge Base (Facts about the Laptop):
  [POWER]
    - turns_on: True
    - charger_connected: False
    - battery_percent: 12
  [PERFORMANCE]
    - cpu_temperature: 92
    - is_slow: True
  [STATUS]
    - fan_noise: True
============================================================
RULE-BASED REASONING (RBR)
============================================================
Knowledge Base (Facts about the Laptop):
  [POWER]
    - turns_on: True
    - charger_connected: False
    - battery_percent: 12
  [PERFORMANCE]
    - cpu_temperature: 92
    - is_slow: True
  [STATUS]
    - fan_noise: True

Rule-Based Reasoning Actions:
- Plug in the charger
- Clean the air vents and cooling fan
- Close heavy programs and run a malware scan
- Turn on battery saver mode
```

CBR code:

```
# ====================================================================
# PART 3 - CASE-BASED REASONING (CBR)
# Task 3A - Case base: past problems and the solutions that worked
# ====================================================================
import sys
from TA4kr import facts, show_knowledge_base
case_base = [
  {"problem": {"wont_turn_on": True,  "noise": False, "overheating": False, "slow": False, "battery_drains_fast": False},
   "solution": "Check power cable."},
```

```
    {"problem": {"wont_turn_on": False, "noise": True,    "overheating": True,    "slow": True,
"battery_drains_fast": False},
     "solution": "Clean the cooling fan and replace thermal paste."},
    {"problem": {"wont_turn_on": False, "noise": False, "overheating": False, "slow": True,
"battery_drains_fast": False},
     "solution": "Free up disk space and restart the laptop."},
    {"problem": {"wont_turn_on": False, "noise": True,    "overheating": False, "slow": False,
"battery_drains_fast": False},
     "solution": "Check for loose parts or a failing fan."},
]


# Task 3B - Retrieve step (similarity matching)
def similarity(new_problem, old_problem):
    """Fraction of features whose values match (0.0 to 1.0)."""
    matches = sum(1 for key in new_problem
                  if old_problem.get(key) == new_problem[key])
    return matches / len(new_problem)


def retrieve(new_problem, cases):
    """Return the most similar case and its similarity score."""
    best_case, best_score = None, -1
    for case in cases:
        score = similarity(new_problem, case["problem"])
        print(f"  Similarity with case -> {score:.2f}  ({case['solution']})")
        if score > best_score:
            best_case, best_score = case, score
    return best_case, best_score


# Task 3C - Reuse step
def reuse(best_case):
    """Reuse the solution of the most similar case."""
    return best_case["solution"]


# Task 3D - Revise step
def revise(solution):
    """Let the student adjust the solution manually, or keep it unchanged."""
    print(f"\nSuggested solution: {solution}")
    answer = input("Type a revised solution (or press Enter to keep it): ").strip()
    if not sys.stdin.isatty():      # echo input when it is piped (for the report)
        print(answer)
```

```
    return answer if answer else solution


# Task 3E - Retain step
def retain(new_problem, final_solution, cases):
    """Add the new case to the case base."""
    cases.append({"problem": new_problem, "solution": final_solution})
    print(f"\nNew case retained. Case base now has {len(cases)} cases.")


def run_cbr():
    print("=" * 55)
    print("CASE-BASED REASONING (CBR)")
    print("=" * 55)

    # New problem to solve (matches the same laptop as the RBR facts)
    new_problem = {"wont_turn_on": False, "noise": True,
                   "overheating": True, "slow": True,
                   "battery_drains_fast": True}
    print("New problem:", new_problem)
    print(f"Case base size before: {len(case_base)}\n")

    print("RETRIEVE:")
    best_case, score = retrieve(new_problem, case_base)
    print(f"  Best match score: {score:.2f}")

    print("\nREUSE:")
    solution = reuse(best_case)
    print(f"  Reused solution: {solution}")

    print("\nREVISE:", end="")
    final_solution = revise(solution)
    print(f"Final solution: {final_solution}")

    print("\nRETAIN:", end="")
    retain(new_problem, final_solution, case_base)
    print()


if __name__ == "__main__":
    run_cbr()
```

```
edrianesagpang@192 TA4 % python3 TA4cbr.py
Knowledge Base (Facts about the Laptop):
  [POWER]
    - turns_on: True
    - charger_connected: False
    - battery_percent: 12
  [PERFORMANCE]
    - cpu_temperature: 92
    - is_slow: True
  [STATUS]
    - fan_noise: True
==================================================
CASE-BASED REASONING (CBR)
==================================================
New problem: {'wont_turn_on': False, 'noise': True, 'overheating': True, 'slow': True, 'batter
_drains_fast': True}
Case base size before: 4

RETRIEVE:
  Similarity with case -> 0.00  (Check power cable.)
  Similarity with case -> 0.80  (Clean the cooling fan and replace thermal paste.)
  Similarity with case -> 0.40  (Free up disk space and restart the laptop.)
  Similarity with case -> 0.40  (Check for loose parts or a failing fan.)
  Best match score: 0.80

REUSE:
  Reused solution: Clean the cooling fan and replace thermal paste.

REVISE:
Suggested solution: Clean the cooling fan and replace thermal paste.
Type a revised solution (or press Enter to keep it):
```

1. Which system was easier to implement? Why?
   - RBR was easier to implement. Each rule is a simple IF-THEN function, and the inference engine is only a loop that
     checks them. CBR needed more parts: a case base, a similarity function, and the four steps (retrieve, reuse, revise,

retain). However, CBR needed no expert to write rules, only past examples.

2. Which is more flexible in solving unstructured problems?

- CBR is more flexible. When a problem does not match any rule, RBR returns nothing. CBR can still find the closest

past case and adapt its solution, even if the problem is only partly similar (in our run, the best match was only 0.80

similar and was revised by the student).

3. Which is better for situations with continuously changing data?

- RBR is better for continuously changing data such as live temperature or battery readings, because the rules are

simply re-checked against the newest facts and the answer updates right away. CBR needs a new case comparison

each time and works better when the situation is described as a whole problem instead of a stream of numbers. CBR

does improve over time because it retains new cases.

4. Real-world applications

- RBR: medical expert systems that follow clinical guidelines, bank loan approval rules, fire alarm and smart-home automation, tax computation software.

CBR: IT help-desk and customer support systems that reuse past tickets, legal systems that search similar past court cases, recipe or product recommendation, and equipment repair advisors.